



Agentic Coding with Antigravity CLI

Google's AI Agent in Your Terminal

Press Space for next page →

Contact Info

Ken Kousen Kousen IT, Inc.

- ken.kousen@kousenit.com
- <http://www.kousenit.com>
- <http://kousenit.org> (blog)
- Social Media:
 - [@kenkousen](https://twitter.com/kenkousen) (twitter)
 - [@kenkousen@foojay.social](https://mastodon.social/@kenkousen) (mastodon)
 - [@kousenit.com](https://kousenit.com) (bluesky)
- *Tales from the jar side* (free newsletter)
 - <https://kenkousen.substack.com>
 - <https://youtube.com/@talesfromthejarside>

Course Overview

- **Duration:** 5 hours of hands-on learning
- **Format:** Instructor-led with multiple labs
- **Hands-on Labs:** Real codebases in Python, JavaScript, Java
- **Prerequisites:** Command-line experience, development background

Topics Covered

- **Foundation:** Installation, CLI basics, authentication
- **Core Skills:** File operations, shell integration, context management
- **Customization:** `GEMINI.md` , `AGENTS.md` , rules, settings
- **Safety:** Sandbox mode, permission prompts, checkpointing
- **Workflow:** Plan, approve, inspect, test, recover, commit
- **Extensions:** MCP, skills/plugins, subagents, session management

Course Through-Line

- Start with a small generated app to learn the interface
- Move quickly into existing codebases
- Ask for a plan before meaningful edits
- Review diffs, run tests, and use checkpoints/rewind
- Finish with a weather API modernization capstone

The 2026 Transition

- Google is moving Gemini CLI users to **Antigravity CLI**
- Consumer/free/AI Pro/AI Ultra Gemini CLI requests stop **June 18, 2026**
- Enterprise Gemini Code Assist access is handled differently
- Antigravity CLI is **not** a 1:1 feature clone at launch
- Core constructs survive: context files, skills, hooks, subagents, MCP, plugins
- Command name changes from `gemini` to `agy`

What is Antigravity CLI?

- Terminal-first surface for Google Antigravity agents
- Launched as the supported successor to Gemini CLI
- Built-in tools: file ops, shell, web/search capabilities
- Model Context Protocol (MCP) support
- Background and parallel subagent workflows
- Designed for developers who live in the terminal

Key Differentiators

- **Unified Harness:** Shares Antigravity's agent platform
- **Terminal-First:** agy keeps agent work in your shell
- **Subagents:** Delegate work to background agent sessions
- **Configurable:** Permissions, keybindings, themes, settings
- **MCP / Skills / Plugins:** Extensible workflows

Antigravity Surfaces

- **Antigravity**: agent-first desktop app for orchestration
- **Antigravity IDE**: developer IDE surface
- **Antigravity CLI**: terminal UI, launched with `agy`
- **Antigravity SDK**: preview path for custom agents
- The course focuses on **Antigravity CLI**

Authentication

- Local use defaults to browser-based Google sign-in
- SSH sessions print a URL and authorization-code flow
- Enterprise use can connect Google Cloud projects
- Logout from the TUI with `/logout`
- First launch also asks for theme, rendering mode, and workspace trust

Installation

- **macOS / Linux:** `curl -fsSL https://antigravity.google/cli/install.sh | bash`
- **Windows PowerShell:** `irm https://antigravity.google/cli/install.ps1 | iex`
- Installs the `agy` binary
- Verify: `agy --version`
- Start inside a trusted project directory

```
1  # Install on macOS / Linux
2  curl -fsSL https://antigravity.google/cli/install.sh | bash
3
4  # Verify installation
5  agy --version
```

First Launch

- Open a project directory
- Run `agy`
- Pick visual theme and rendering mode
- Confirm workspace trust
- Complete Google sign-in

```
1  cd my-project
2  agy
```


Basic Usage Modes

- **Interactive REPL:** `agy` - Start a conversation
- **One-shot:** `agy "prompt"` - Single response
- **Piped input:** `echo "task" | agy`
- **Interactive with context:** `agy -i "initial context"`

```
1  # Interactive mode
2  agy
3
4  # One-shot mode
5  agy "Explain what this codebase does"
6
7  # With initial context
8  agy -i "You are a Python expert"
```

Core Features

Essential Capabilities

Master the fundamentals



File References with @

- Reference files directly: `@./src/main.js`
- Reference directories: `@./src/` (recursive)
- Reference images: `@./screenshot.png`
- Multiple references in one prompt

```
1  # Reference a specific file
2  agy "Explain @./src/app.py"
3
4  # Reference multiple files
5  agy "Compare @./old.js and @./new.js"
6
7  # Reference a directory
8  agy "Analyze the architecture in @./src/"
```

Shell Integration with !

- Execute shell commands: `!git status`
- Toggle persistent shell mode: `!`
- Antigravity can observe and analyze output
- Combine with AI analysis

```
1  # In interactive mode:
2  > !npm test
3  # Antigravity sees the test output
4
5  > !git diff
6  # Ask Antigravity to analyze the changes
7
8  # Toggle persistent shell mode
9  > !
```

Slash Commands: Navigation

- `/help` - Show available commands
- `/clear` - Clear conversation history
- `/config` - Open settings
- `/permissions` - Manage tool permissions

Slash Commands: Workflow

- `/agents` - Open subagents panel
- `/fork` - Branch from an earlier point
- `/logout` - Remove saved credentials

Slash Commands: Session Control

- `/resume` - Open session browser
- `/rewind` - Navigate and optionally revert history
- `/restore` - Recover from checkpoint

Slash Commands in Action (Context)

```
1  # Open settings and permissions
2  /config
3  /permissions
4
5  # Inspect subagents
6  /agents
7
8  # Ask about active context
9  What context files and rules did you load?
```


Slash Commands in Action (Sessions)

```
1  # Open session browser
2  /resume
3
4  # Rewind through recent interactions
5  /rewind
6
7  # Restore from checkpoint list
8  /restore
```

Keyboard Shortcuts: Editing

- `Ctrl+L` - Clear screen
- `Ctrl+V` - Paste text/images
- `Ctrl+G` - Open external editor (was `Ctrl+X` before 0.37)

Keyboard Shortcuts: Control

- Use the built-in help screen for current keybindings
- Treat auto-approval shortcuts as version-specific
- `Ctrl+C` - Cancel current operation
- `Ctrl+D` - Exit Antigravity CLI

Built-in Tools

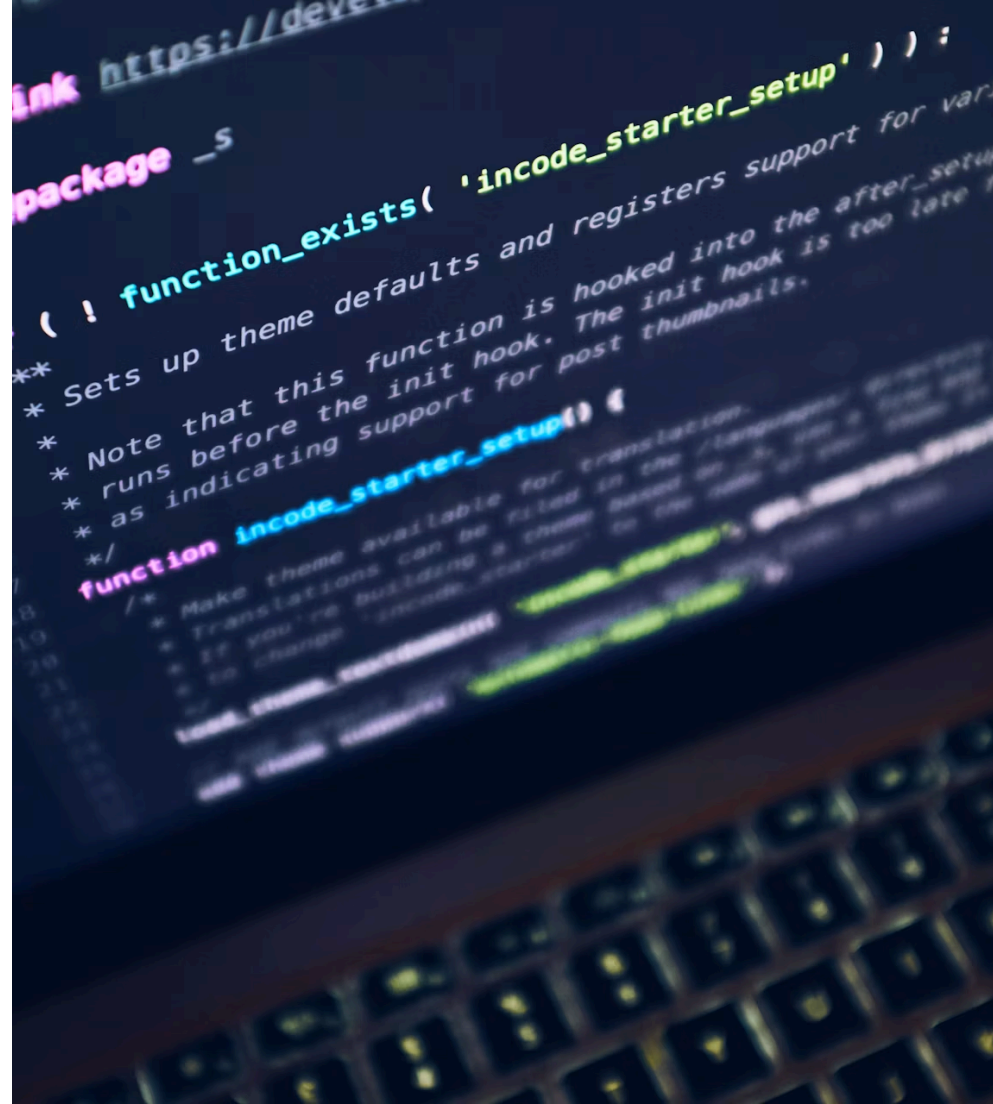
- **File System:** `read_file()`, `write_file()`, `replace()`, `glob()`
- **Shell:** Execute terminal commands
- **Web:** `google_web_search()`, `web_fetch()`
- **Memory:** `save_memory()` for cross-session recall

```
1  # Antigravity automatically uses appropriate tools
2  "Search the web for React 19 new features"
3  # Uses google_web_search()
4
5  "Read all Python files in src/"
6  # Uses glob() and read_file()
7
8  "Update the README with the changes we made"
9  # Uses write_file()
```

Safety & Control

Work Safely

Approval modes and sandboxing



Permissions

- Antigravity CLI prompts before sensitive tool use
- Use `/permissions` to review and adjust what agents may do
- Use `/config` or `/settings` for broader safety behavior
- `--sandbox` isolates execution where supported
- `--dangerously-skip-permissions` is for disposable demos only

```
1  agy
2
3  /permissions
4  /config
```

Plan Mode

- Ask for a plan before approving file edits or commands
- Review the intended files, tests, and fallback path
- Open longer prompts in your external editor with `Ctrl+G`
- Use `/rewind` or `/undo` when the conversation drifts
- Use `/fork` to branch from a useful earlier point
- Great first step when exploring an unfamiliar codebase

```
1  Plan the change first. Do not edit files yet.
```

Sandbox Mode

- Isolate file operations away from your host
- Multiple backends depending on your OS
- Prevents accidental system changes
- Perfect for exploring unfamiliar code

```
1  # Run in sandbox mode (picks the best backend for your OS)
2  agy --sandbox
3
4  # Short form
5  agy -s "Refactor this entire codebase"
```


Sandbox Mode

- Use sandbox mode when letting agents run commands
- Combine with small, reviewable tasks
- Still inspect generated changes
- Sandbox behavior depends on platform support
- Good for unfamiliar repos, generated tests, and refactors

```
1 agy --sandbox
```

 antigravity.google/docs/cli-settings

Checkpointing

- **Automatic:** Snapshots created before each file modification
- **Session History:** Resume and rewind are built into the TUI
- **Includes:** Files + conversation + tool call
- **Disabled by default:** Must enable in settings

```
1 // ~/.gemini/antigravity-cli/settings.json
2 { "general": { "checkpointing": { "enabled": true } } }
```

Restoring Checkpoints

```
1  # List and select a checkpoint to restore
2  /restore
3
4  # Shows timestamps + filename + tool name
5  # e.g., 2025-06-22T10-00-00_000Z-app.py-write_file
```

Restores files AND resets conversation to that point



Context Management

Context Files and Rules

Project memory and instructions

Context Files

- **Workspace context** loaded automatically
- **Coding standards** and conventions
- **Architecture context** for the AI
- **Persistent instructions** across sessions
- Antigravity CLI reads both `GEMINI.md` and `AGENTS.md`
- Workspace rules can also live under `.agents/rules/`

Hierarchical Loading

Example AGENTS.md

```
1  # Project: Weather API
2  ## Tech Stack
3  - Backend: Python Flask
4  - Database: PostgreSQL
5  - Testing: pytest
6  ## Coding Standards
7  - Use type hints for all functions
8  - Follow PEP 8 style guide
9  - Write docstrings for public APIs
10 ## Current Focus
11 Implementing caching layer for API responses
```

Context Commands

- `/config` - Manage settings
- `@` - Reference files and rules directly
- `/rewind` - Back out of context drift
- `/fork` - Branch from a useful earlier point
- Ask the agent what context it loaded

```
1  What project rules and context files are active right now?
```


Modular Imports

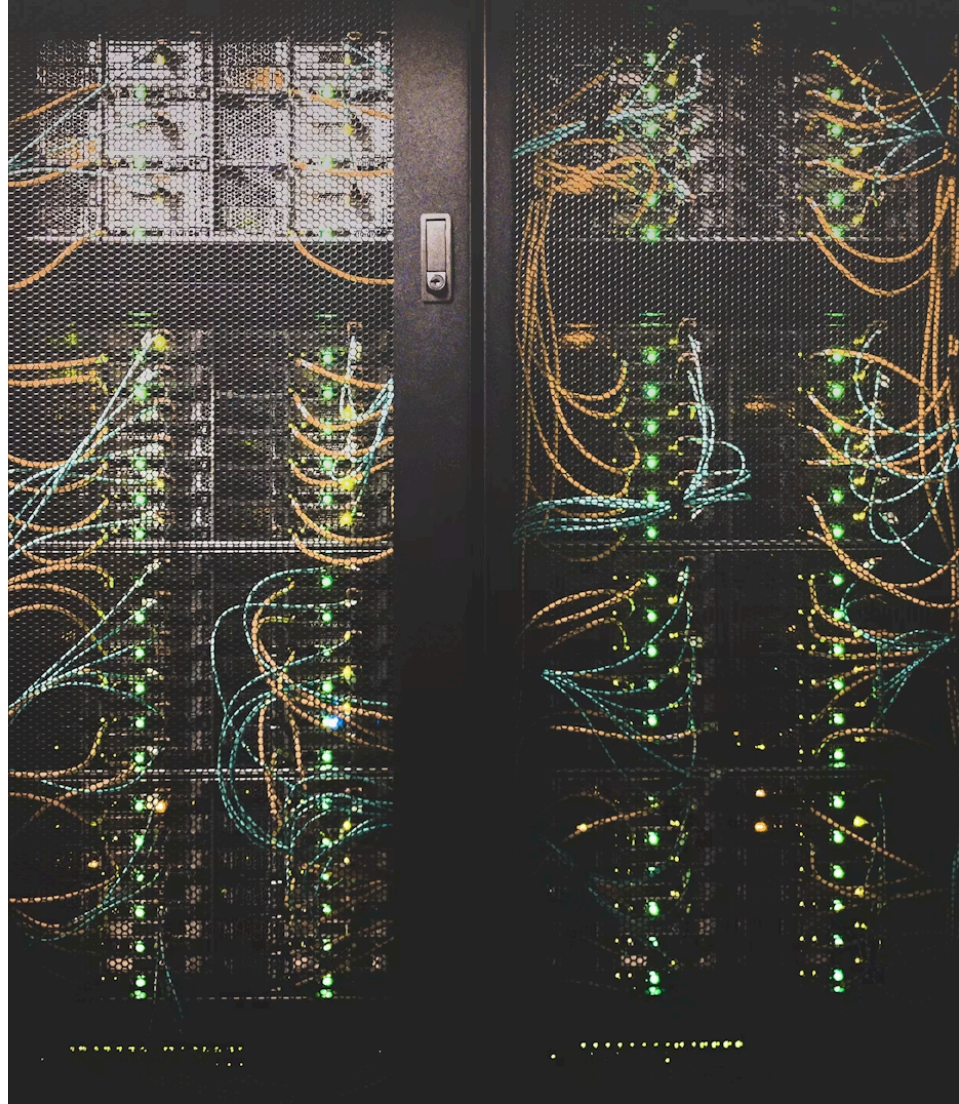
- Import other files with `@file.md` syntax
- Break large context into components
- Supports relative and absolute paths

```
1  # AGENTS.md
2
3  ## Project Overview
4  @./docs/architecture.md
5
6  ## Coding Standards
7  @./docs/style-guide.md
8
9  ## API Documentation
10 @./docs/api-reference.md
```

Configuration

Customize Your Setup

settings.json and environment



Configuration Files

1. **Settings** - `~/.gemini/antigravity-cli/settings.json`
2. **Keybindings** - `~/.gemini/antigravity-cli/keybindings.json`
3. **Global rules** - `~/.gemini/GEMINI.md`
4. **Workspace rules** - `.agents/rules/`
5. **Workspace skills** - `.agents/skills/`

settings.json Options

```
1  {  
2    "general": {  
3      "preferredEditor": "vscode",  
4      "vimMode": false,  
5      "checkpointing": { "enabled": true }  
6    }  
7  }
```

settings.json Options (UI + Tools)

```
1  {  
2    "ui": {  
3      "hideTips": false,  
4      "hideBanner": false  
5    },  
6    "tools": {  
7      "sandbox": false  
8    }  
9  }
```

Tool Permissions

- Use `/permissions` for interactive permission management
- Use `/config` or `/settings` for persistent settings
- Keep high-risk commands behind prompts in teaching demos
- Prefer sandbox mode for generated shell work

```
1  /permissions
2  /config
```

 antigravity.google/docs/cli-permissions

File Filtering

- **respectGitIgnore:** Honor .gitignore patterns
- **enableRecursiveFileSearch:** Recursive completion
- **Rules:** Keep generated/sensitive paths out of agent context

```
1  {  
2    "context": {  
3      "fileFiltering": {  
4        "respectGitIgnore": true,  
5        "enableRecursiveFileSearch": true  
6      }  
7    }  
8  }
```

Environment Variables

Variable	Purpose
Browser sign-in	Default local authentication
<code>GOOGLE_API_KEY</code>	API-key route when supported
<code>GOOGLE_CLOUD_PROJECT</code>	GCP project for Vertex AI
<code>GOOGLE_CLOUD_LOCATION</code>	GCP region
<code>HTTP_PROXY</code>	Network proxy

Context File Strategy

- Use `AGENTS.md` for cross-tool repository rules
- Keep `GEMINI.md` for Antigravity/Gemini-specific context
- Put reusable workspace rules under `.agents/rules/`
- Put workspace skills under `.agents/skills/`

```
1  project/  
2  |— AGENTS.md  
3  |— GEMINI.md  
4  |— .agents/  
5     |— rules/  
6     |— skills/
```

Advanced Features

Optional Extension Tools

MCP, Skills, Plugins, Sessions



Advanced Features: Teaching Stance

- Useful after students understand the core agent loop
- Demonstrate one MCP or skill workflow, not every option
- Keep configuration examples version-checked with `agy --help`
- Treat subagents and plugins as follow-up practice when time is short

Model Context Protocol (MCP)

- Standard protocol for AI-to-system connections
- Connect to external tools and services
- Supports local commands, HTTP, and SSE
- OAuth 2.0 for remote authentication

MCP Configuration: Firecrawl

```
1  {
2    "mcpServers": {
3      "firecrawl": {
4        "command": "npx",
5        "args": ["@modelcontextprotocol/server-firecrawl"],
6        "env": {
7          "FIRECRAWL_API_KEY": "${FIRECRAWL_API_KEY}"
8        }
9      }
10   }
11 }
```

MCP Configuration: Database

```
1  {  
2    "mcpServers": {  
3      "postgres": {  
4        "command": "npx",  
5        "args": ["@modelcontextprotocol/server-postgres"],  
6        "env": {  
7          "CONNECTION_STRING": "${DATABASE_URL}"  
8        }  
9      }  
10   }  
11 }
```

Managing MCP Servers

```
1  # List configured MCP servers
2  agy mcp list
3
4  # Add a new MCP server
5  agy mcp add github npx -y @modelcontextprotocol/server-github
6
7  # Remove an MCP server
8  agy mcp remove github
```

Managing MCP Auth/Refresh

```
1  # List MCP tools/resources from inside session
2  /mcp list
3
4  # Authenticate with OAuth-enabled MCP server
5  /mcp auth github
6
7  # Refresh MCP tools/resources
8  /mcp refresh
```


MCP Server Options

- **command**: Shell command to start server
- **args**: Command arguments
- **env**: Environment variables
- **cwd**: Working directory

MCP Server Options (Advanced)

- **timeout**: Startup timeout in ms
- **includeTools/excludeTools**: Filter available tools
- **trust**: Bypass tool confirmation for this server

Popular MCP Servers

- **Firecrawl** - Web scraping, search, content extraction
- **PostgreSQL** - Database queries and schema
- **Filesystem** - Extended file operations
- **Slack** - Team communication
- **Playwright** - Browser automation

📖 **Registry:** modelcontextprotocol.io/registry

Plugins and MCP

- Antigravity CLI separates MCP server config from general settings
- Gemini CLI `settings.json` MCP entries migrate to `mcp_config.json`
- Some Gemini CLI extensions become Antigravity plugins
- Not every extension migrates 1:1

```
1 agy mcp list
```

 antigravity.google/docs/gcli-migration

A2A Remote Agents (0.33+)

- Antigravity CLI is built for multi-agent workflows
- Use `/agents` to open the agent panel
- Delegate background work to subagents
- Monitor status without blocking your terminal flow

Extensions System

- Extend Antigravity with plugins and skills
- Global CLI skills live under `~/.gemini/antigravity-cli/skills/`
- Workspace skills live under `.agents/skills/`
- Reusable prompts belong in skills/workflows

```
1  /agents  
2  /config
```

Reusable Skills

- Skills package repeatable agent behavior
- Store workspace skills in `.agents/skills/<name>/SKILL.md`
- Keep descriptions specific so agents know when to use them
- Use skills for review, test generation, docs, refactoring

```
1  ---
2  name: code-review
3  description: Review code for issues.
4  ---
5
6  Review code for:
7  - Security vulnerabilities
8  - Performance issues
9  - Best practices violations
```

Session Management

- **Resume sessions:** Continue previous conversations
- **Auto-save resume:** CLI prints a resume command on exit
- **Session browser:** `/resume` from inside the TUI
- **Rewind:** `/rewind` or `/undo` backs out of drift

```
1 agy
2 /resume
3 /rewind
```


Keybindings

- `/keybindings` opens the shortcut editor
- `Ctrl+G` opens your prompt in `$EDITOR`
- `Ctrl+D` exits the TUI
- `Ctrl+L` clears the screen
- `Ctrl+C` or `Esc` cancels/backs out

```
1 /keybindings
```

IDE Integration

- **VS Code Integration:** Connect to workspace
- **Native diff viewing:** Review changes in editor
- **Context sharing:** IDE context available to Antigravity agents

```
1  {  
2    "preferredEditor": "vscode"  
3  }
```



Practical Applications

Real-World Workflows

Common use cases

Code Exploration

- Understand unfamiliar codebases
- Trace dependencies and data flow
- Find patterns and conventions
- Generate architecture documentation

```
1 agy "Analyze the architecture of @./src/ and explain  
2 how the components interact"  
3  
4 agy "Trace the flow from the API endpoint to the database  
5 in @./src/controllers/ and @./src/services/"
```

Test Generation

- Generate unit tests for existing code
- Identify edge cases automatically
- Create integration test scaffolding
- Mock setup and fixtures

```
1 agy "Create comprehensive unit tests for @./src/utils.py
2   with pytest, including edge cases"
3
4 agy "Generate integration tests for @./src/api/users.py
5   with proper mocking"
```

Documentation Generation

- README files for projects
- API documentation
- Architecture diagrams (Mermaid)
- Code comments and docstrings

```
1 agy "Generate a comprehensive README.md for this project"
2
3 agy "Add detailed docstrings to all public functions
4 in @./src/services/"
5
6 agy "Create a Mermaid diagram showing the system architecture"
```

Refactoring & Modernization

- Upgrade legacy code patterns
- Apply modern language features
- Improve code organization
- Fix anti-patterns

```
1 agy "Refactor @./src/legacy.py to use modern Python 3.12  
2 features like type hints and match statements"  
3  
4 agy "Convert this callback-based code to async/await  
5 @./src/api.js"
```

Debugging Workflows

- Analyze error messages and stack traces
- Identify root causes
- Suggest fixes with context
- Test and verify solutions

```
1 agy "This test is failing with @./tests/output.log.  
2 Analyze the error and fix the issue in @./src/app.py"  
3  
4 agy "Debug why the API returns 500 errors.  
5 Check @./src/routes.py and @./src/middleware.py"
```


Capstone: Weather API Modernization

- Audit the README, Swagger docs, routes, services, and tests
- Make offline behavior intentional and testable
- Add a small caching or refactoring improvement
- Run unit, integration, and browser/e2e tests
- Inspect the diff and create a clear commit

```
1  cd exercises/python/weather-app
2  agy
```

Git Workflows

- Generate commit messages
- Create pull request descriptions
- Analyze diffs and changes
- Resolve merge conflicts

```
1  # Analyze staged changes
2  !git diff --staged
3  "Generate a conventional commit message for these changes"
4
5  # Create PR description
6  "Create a pull request description summarizing the changes
7  from the last 5 commits"
```

CI/CD Integration

- Use the CLI only where current `agy --help` confirms support
- Keep CI prompts deterministic and repo-scoped
- Store logs/artifacts for auditability
- Prefer human review for code-changing workflows

```
1 agy --help
2 agy "Review @./src/ for security issues"
```

Optional Advanced Module

- Designed for teams moving from personal usage to production workflows
- Can be taught live or assigned as follow-up practice
- Focus areas: auth strategy, governance, skills/MCP, CI automation



Authentication Matrix (Team Use)

- **Login with Google:** Best default for local interactive usage
- **Browser sign-in:** Best default for local interactive usage
- **Google Cloud project:** Enterprise onboarding path
- **ADC / service account:** CI and controlled automation where supported
- **June 18, 2026:** Gemini CLI consumer transition deadline

Auth Quick Checks

```
1  # Local route
2  agy
3
4  # Cloud route (ADC)
5  unset GOOGLE_API_KEY
6  gcloud auth application-default login
7  export GOOGLE_CLOUD_PROJECT="my-project"
8  export GOOGLE_CLOUD_LOCATION="us-central1"
9  agy
```

Governance: Approval and Policies

- Approval mode controls are per-session safety rails
- Policies provide durable guardrails across users/projects
- Keep high-risk tools constrained in team settings
- Prefer explicit allow/deny patterns over ad-hoc approvals

Governance: Hooks and Trust

- `/hooks list` shows active lifecycle hooks
- Use hooks for auditing, validation, and policy enforcement
- Folder trust impacts settings, skills, and context loading
- Treat untrusted repos with stricter modes and sandboxing

Skills + MCP at Scale

- Skills package repeatable expert workflows for teams
- MCP connects external systems (docs, data, browsers, APIs)
- Resource URIs can be injected via `@server://resource/path`
- Use `includeTools` / `excludeTools` to narrow risky MCP exposure

Skills + MCP Commands

```
1  # Skills lifecycle
2  /skills list
3  /skills disable skill-name
4  /skills enable skill-name
5
6  # MCP visibility and maintenance
7  /mcp list
8  /mcp auth server-name
9  /mcp refresh
```

Automation Patterns (Non-Interactive)

- Verify current non-interactive flags with `agy --help`
- Use exit codes only after testing the exact command
- Keep prompts deterministic and repo-scoped
- Store logs/artifacts for auditability

CI Example: Gate + Report

```
1 agy --help
2 agy "Review @./src/ for security issues and write a concise report"
```

Best Practices

Professional Workflows

Tips for success



Effective Prompting

- **Be specific** about what you want to achieve
- **Provide context** about goals and constraints
- **Use file references** to ground the conversation
- **Iterate** for complex tasks
- **Include examples** when possible

Safety Guidelines

- **Plan first** - Ask what will change before approving edits
- **Enable checkpointing** - Safety net for mistakes
- **Review generated code** - Inspect the diff before committing
- **Use sandbox** - Helpful for exploratory work
- **Run tests and commit** - Make progress auditable

Project Setup

1. Create concise `AGENTS.md`
2. Add Antigravity-specific `GEMINI.md` only when needed
3. Configure relevant MCP servers
4. Create `.agents/skills/` for common workflows
5. Establish team conventions

Context Best Practices

- **Keep it focused** - Relevant project info only
- **Update regularly** - Reflect current state
- **Prefer AGENTS.md** - Shared rules across tools
- **Include examples** - Show expected patterns
- **Document conventions** - Style guides, patterns

Team Collaboration

- Share `AGENTS.md` in version control
- Document Antigravity settings for the team
- Create shared skills, rules, and workflows
- Document AI-assisted workflows
- Review AI-generated code together

Common Pitfalls

- **Overly broad prompts** → Be specific
- **Missing context** → Use AGENTS.md or workspace rules
- **Skipping review** → Always verify output
- **Skipping permissions** → Only for disposable demos
- **Ignoring checkpoints** → Enable early

Troubleshooting

- **Authentication issues:** Re-run browser sign-in or `/logout`
- **Rate limits:** Use appropriate tier
- **Tool failures:** Check MCP server status
- **Context not loading:** Ask which files/rules were loaded
- **CLI flags:** Use `agy --help` for the current reference

```
1  # Debug mode for troubleshooting
2  agy --help
3
4  # Inspect loaded context
5  What context files and rules did you load?
6
7  # Review active settings and permissions
8  /config
9  /permissions
```

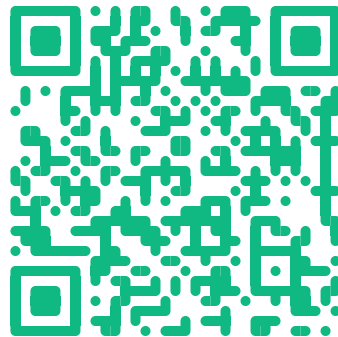
Quick Access

Antigravity CLI Docs



antigravity.google/docs/cli-getting-started

Course Repository



github.com/kousen/gemini-training

Important Links



Official Documentation

<https://antigravity.google/docs/cli-getting-started>



Migration Guide

<https://antigravity.google/docs/gcli-migration>



MCP Server Registry

<https://modelcontextprotocol.io/registry>



Course Materials

<https://github.com/kousen/gemini-training>

Command Reference: Basic Usage

```
1  # Interactive mode
2  agy
3
4  # One-shot mode
5  agy "prompt"
6
7  # Interactive with initial context
8  agy -i "context"
```

Command Reference: Safety

```
1  # Run in sandbox mode
2  agy --sandbox
3
4  # Disposable demo only
5  agy --dangerously-skip-permissions
```


Command Reference: Sessions

```
1 agy
2 /resume
3 /rewind
```

Command Reference: Output

```
1 agy --help
2 agy "prompt"
```

Thank You!

Questions?



Kenneth Kousen *Author, Speaker, Java & AI Expert*

kousenit.com | [@kenkousen](https://twitter.com/kenkousen)